

IADS Agentic AI Hackathon 2026

Multi-Stage Text-to-SQL Agent

on Oracle Cloud Infrastructure

Team 4 · Project Briefing & Task Allocation

Member	Background	Slice
Omar	AI	Orchestration, critic, model routing
Hasan	AI	Planner, SQL generator, LLM client
Zayad	AI	Retrieval / RAG layer
Asad	Data Science (MSc)	Summariser, evaluation, data modelling
Mehdi	CS (undergrad)	FastAPI service, Streamlit UI
Abdul Qayyum	CS (undergrad)	Database, vector store, safety

Hackathon dates: 2-4 June 2026 · Topic: Natural Language Query Agent (Text-to-SQL)

Contents

#	Section	Page
1.	How to read this document	p. 3
2.	Project at a glance	p. 4
3.	Architecture — the 7-stage pipeline	p. 5
4.	Repository map	p. 7
5.	GitHub workflow	p. 9
6.	Day 1 morning — the alignment session	p. 11
7.	Per-person briefings	p. 12
7.1	Omar — Orchestration & Critic	p. 13
7.2	Hasan — Planner & SQL Generator	p. 16
7.3	Zayad — Retrieval / RAG	p. 19
7.4	Asad — Summariser, Evaluation & Data	p. 22
7.5	Mehdi — FastAPI & Streamlit	p. 26
7.6	Abdul Qayyum — Database & Vector Store	p. 29
8.	Integration points	p. 32
9.	Request lifecycle	p. 33
10.	Day-by-day timeline	p. 35
11.	Demo plan	p. 37
12.	Appendices	p. 38

1. How to read this document

This briefing is the single source of truth for what each person builds during the hackathon. Read it once front to back; then read your own section in detail before the kickoff.

What you'll find here

- **Sections 2-5** — shared context everyone needs: what we're building, the architecture, the repository map, and how we collaborate via GitHub.
- **Section 6** — the Day 1 alignment session. Nothing else can start until we run this together.
- **Section 7** — six per-person briefings. Each is 3 pages: your mission, your files, the functionalities you implement, your Pydantic contracts, your dependencies, and your Day 1/2/3 plan.
- **Sections 8-11** — how the slices fit together, the request lifecycle, the timeline, and the demo plan.
- **Section 12** — appendices: full ownership matrix, environment setup checklist, cheat sheet.

How to use it

- **Tonight:** read your section twice. Note anything unclear and raise it before the kickoff.
- **Day 1 morning:** we meet for the alignment session (Section 6). Bring your section with you.
- **During the hackathon:** when you finish a file, push it. When you start one, tell the group chat. The briefing tells you what each file should do, not how the code should look.
- **If something blocks you:** the Dependencies sub-section of your briefing tells you who to ask.

Important: This document describes the skeleton the repository already contains. Every file mentioned exists as a placeholder with a docstring describing the public interface. Your job is to fill in the implementation that matches the docstring.

2. Project at a glance

What we are building

A natural-language data analyst. A user types a business question in plain English; the system returns a one-sentence answer, the underlying result table, and the SQL that produced both. The system is built as a five-stage agentic pipeline on Oracle Cloud Infrastructure, using OCI Generative AI for the LLM calls and Oracle 23ai for vector retrieval.

Why this topic

Text-to-SQL is currently the highest-leverage application of LLMs in enterprise data: every company with a database has a thousand analysts manually translating questions into SQL every week. A working agent that produces measurable accuracy on a benchmark set is a strong CV story for every team member and a credible piece of work for the judges.

The brief's three explicit requirements

- **RAG** — the system retrieves relevant context (schema metadata) from a vector store before generating SQL.
- **OCI AI services** — we use OCI Generative AI for LLM calls and OCI Generative AI Embeddings for vectorisation.
- **Deployed AI-enabled service** — the system runs end-to-end as a FastAPI backend plus a Streamlit frontend, packaged in Docker.

Six features that go beyond the baseline

These differentiate us from any team that submits a single-shot prompt-to-SQL demo. Each is wired into a specific module in the codebase.

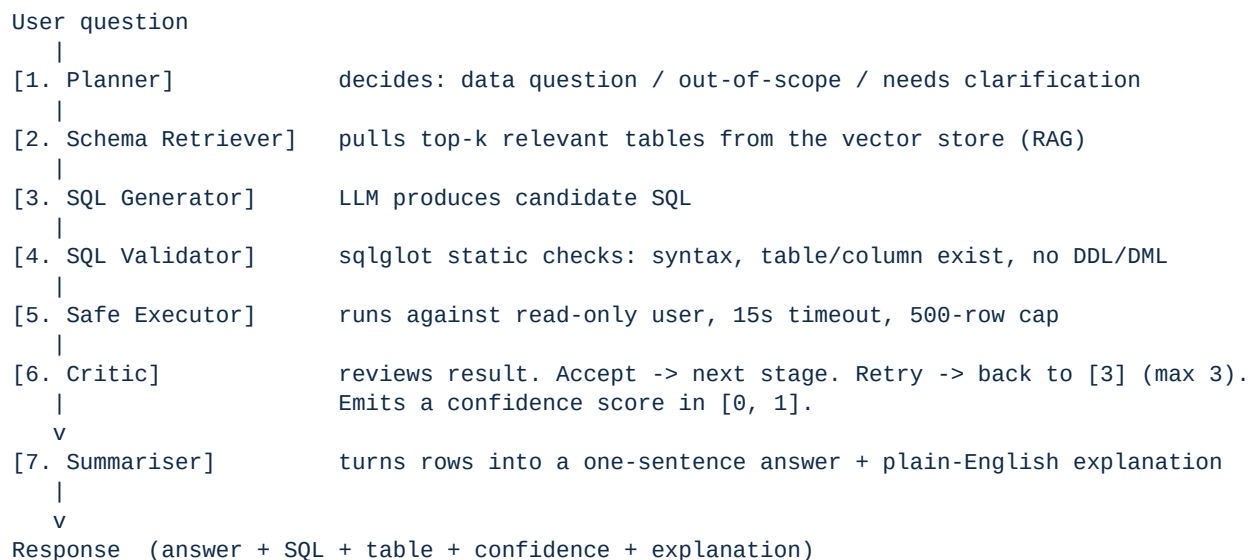
#	Feature	Module
1	Multi-stage agentic pipeline with critic-driven retry	agents/orchestrator + critic
2	Schema-aware RAG over column-level descriptions	retrieval/schema_retriever
3	Dynamic few-shot injection from a vector example bank	retrieval/few_shot_bank
4	Schema obfuscation for privacy-preserving LLM calls	safety/schema_obfuscator
5	Multi-model routing — cheap model for simple queries	llm/model_router
6	Semantic row fallback when SQL returns zero rows	retrieval/row_fallback

Plus two product-grade UI safeguards: confidence-score warning banner, and friendly error states (never raw tracebacks).

Plus a demo-day fail-safe: a hardcoded cache of rehearsed answers that activates only if the live infrastructure drops mid-presentation.

3. Architecture — the 7-stage pipeline

Every user question flows through seven stages. Each stage has a single responsibility and a typed Pydantic contract for its input and output. This means each stage can be developed, tested, and improved independently — which is what makes parallel work across six people possible.



Stage-by-stage contracts

Stage	Owner	Input	Output
Planner	Hasan	Question	Plan
Schema Retriever	Zayad	Question	RetrievedSchema
SQL Generator	Hasan	Question + Schema + Examples	CandidateSQL
SQL Validator	Omar	CandidateSQL + Schema	ValidationResult
Safe Executor	Abdul Qayyum	ValidatedSQL	ExecutionResult
Critic	Omar	Everything-so-far	CritiqueResult (+ confidence)
Summariser	Asad	Question + SQL + Rows	AnswerSummary

Why so many stages?

A single-shot prompt that asks the LLM to 'write SQL' fails on three predictable failure modes: hallucinated table names, valid SQL that runs but answers the wrong question, and silent failures with no observability. The multi-stage design catches each one cheaply: the validator rejects hallucinated columns before they cost a DB round-trip; the critic catches semantic drift; the confidence score surfaces low-confidence answers to the user instead of presenting them as definitive.

Cross-cutting modules

Three modules sit alongside the pipeline rather than as stages within it. Each is small but turns the system from a prototype into something a judge can take seriously:

- **Schema Obfuscator** (safety/schema_obfuscator.py) — renames real tables to Table_A before sending the schema to the LLM, then maps the generated SQL back to real names before execution. Demonstrates enterprise-grade privacy design.
- **Model Router** (llm/model_router.py) — routes simple questions to a cheaper, faster model. Saves tokens and latency, gives us a real number to put on the slide.
- **Few-Shot Bank** (retrieval/few_shot_bank.py) — pulls 2-3 similar past queries from the vector store and injects them into the prompt. The same vector infrastructure as schema retrieval, in a different namespace.

Two fallback paths

- **Row Fallback** (retrieval/row_fallback.py) — when a valid SQL query returns zero rows because of a slight keyword mismatch, embed the filter value and pull semantically similar rows instead. Addresses the brief's explicit 'vector-based decision' requirement.
- **Demo Cache** (database/demo_cache.py) — rehearsal answers for the three demo questions. Activated by an env flag right before the demo, served instantly if the live DB or OCI is unreachable. Insurance only.

4. Repository map

The repository is laid out by layer, not by feature. Each directory has a single concern; the dependency graph runs top-down with no cycles. The full breakdown lives in docs/STRUCTURE.md — here is the orientation.

```
IADS-Agentic-SQL-Agent/
|-- src/sql_agent/      Core Python package (the system itself)
|   |-- core/          Domain models (Pydantic), exceptions, logging
|   |-- config/        Typed settings from environment variables
|   |-- llm/           OCI GenAI client, model router, token counter
|   |-- retrieval/     Embeddings, vector store, schema retriever,
|                       few-shot bank, row fallback
|   |-- database/      Connection pool, safe executor, demo cache,
|                       schema introspector
|   |-- safety/        SQL guard, PII filter, schema obfuscator
|   |-- agents/        Planner, generator, validator, critic,
|                       summariser, orchestrator
|   |-- api/           FastAPI app, routes, HTTP schemas
|
|-- frontend/          Streamlit UI
|-- prompts/           Versioned prompt templates (Markdown)
|-- db/               DDL, migrations, seed data, schema descriptions
|-- evaluation/        Benchmark harness + golden + demo datasets
|-- tests/            Unit + integration tests
|-- scripts/          Operational scripts (seed DB, embed schema, run benchmark)
|-- notebooks/        Exploration and prompt iteration
|-- docs/             Architecture, ADRs, evaluation methodology, lifecycle
|-- data/raw|processed Local datasets (gitignored content)
|-- deploy/           Deployment configuration
|-- observability/    OTel / Prometheus / Grafana config
|-- examples/         Runnable usage examples
|-- .github/          CI workflow, issue + PR templates
```

Where do I put my new file?

You want to add...	Put it in...
A domain model used by multiple modules	src/sql_agent/core/models.py
A new agent stage	src/sql_agent/agents/<stage>.py + entry in orchestrator
A new prompt	prompts/<name>.md
A configuration knob	config/settings.py + .env.example
An operational one-off	scripts/<name>.py + Makefile target
An exploratory analysis	notebooks/NN_<name>.ipynb
A new eval query	append to evaluation/datasets/golden_queries.jsonl
An architectural choice	new ADR in docs/decisions/
A test for a module	mirror the path in tests/unit/

What does not go in this repo

- Secrets, credentials, OCI keys — use .env (gitignored).
- Trained model weights or large embeddings — store in OCI Object Storage, reference by URI.
- Generated artifacts (build/, __pycache__, coverage reports) — gitignored.
- Personal scratch work — keep on a feature branch.

5. GitHub workflow

The whole team works on the same repository simultaneously. The conventions below exist so we don't spend the last six hours of the hackathon resolving merge conflicts. Treat them as non-negotiable.

Branches

- main is protected. No direct pushes. Every change lands via pull request.
- Each person works on their own branch, named `feat/<name>-<module>`. Examples: `feat/omar-orchestrator`, `feat/asad-summariser`.
- Keep branches short-lived — rebase or merge from main every few hours.
- If you need a separate exploration, prefix with `spike/` — e.g. `spike/zayad-cohere-embed-comparison`.

Commits

Use Conventional Commits. The prefix tells everyone what the commit is for.

```
feat(agents):    add SQL validator with sqlglot checks
fix(database):   handle connection timeout gracefully
docs(architect): document the retry loop
test(validator): add cases for hallucinated columns
refactor(llm):   extract prompt loader into its own module
chore(ci):       bump ruff to 0.4.4
```

Allowed prefixes: feat, fix, docs, test, refactor, chore.

Pull requests

- Open a PR as soon as you have something runnable, even if rough. Mark it *Draft*.
- PR template is auto-filled — fill in **What / Why / How / Verification**.
- Tick the verification checklist: make test, make lint, benchmark unchanged or improved.
- Any teammate can approve. CI must be green before merge.
- Squash-merge into main. The PR title becomes the commit message.

Push cadence

Push to your branch every time you finish a logical chunk — not at the end of the day. This means the rest of the team can see your work in progress and pull it for integration tests.

- Minimum once every 2 hours during active work.
- Always push before standups (every 4 hours).
- Always push before going to bed. Always.

Conflict prevention

Most conflicts happen on three files: `core/models.py`, `orchestrator.py`, `settings.py`. The conventions:

- `core/models.py` is frozen after the Day 1 morning alignment. Changes require team chat.
- `orchestrator.py` is Omar's file. If you need a hook in it, ask him.
- `settings.py` — if you need a new env var, add it to `.env.example` in your PR, and mention it in the PR description.

CI

`.github/workflows/ci.yml` runs `ruff` (lint), `mypy` (types), `pytest` (tests), and a quick import smoke-check on every push. If CI is red, fix it before going to bed — never merge into main with red CI.

Standups

Every 4 hours during the hackathon. 5 minutes maximum. Each person says:

- 1 What I just finished.
- 2 What I'm about to do next.
- 3 What's blocking me.

Omar runs the standup. Nobody is allowed to interrupt; questions go into the group chat afterwards.

6. Day 1 morning — the alignment session

Before anyone writes implementation code, the team meets for 60-90 minutes to agree on the Pydantic models in `src/sql_agent/core/models.py`. These models are the typed contracts between every stage of the pipeline. Once they are agreed and pushed to main, every other person can work on their slice in parallel without stepping on each other.

Why this matters: *If we skip this step, every PR will need to renegotiate interfaces while other people are building against them. The two hours we spend here saves ten hours of integration pain.*

Models to agree on

Model	Carries	Used by
Question	The raw user input string + trace ID	Everyone
Plan	decision, scope tags, refusal_reason	Planner -> Orchestrator
RetrievedSchema	list[Table] with descriptions + columns	Retriever -> Generator
FewShotExample	nl_question, sql, rationale	Few-shot bank -> Generator
AliasMap	real_name <-> alias mappings	Obfuscator -> Generator/Validator
CandidateSQL	sql string, model_id used, prompt token count	Generator -> Validator
ValidationResult	valid: bool, error: str None	Validator -> Executor
ExecutionResult	rows, columns, row_count, latency_ms	Executor -> Critic
CritiqueResult	decision: accept retry, feedback, confidence	Critic -> Orchestrator
RowFallbackResult	approximate rows, similarity scores	Row fallback -> Critic
AnswerSummary	answer, explanation, sql, tables_used, confidence	Summariser -> API
QueryRequest	question (HTTP payload)	API
QueryResponse	wraps AnswerSummary + warnings + error	API -> Frontend

Process

- 1 Omar opens a shared screen with `core/models.py`.
- 2 Each owner proposes the fields for their stage's output model. Two minutes per stage.

- 3 The team agrees on naming, types, and default values. No code yet — just the shape.
- 4 Omar writes the Pydantic classes, pushes to main. Everyone pulls.
- 5 Each person opens their own file and writes function signatures that import these models. No implementation yet — just signatures.
- 6 Once signatures are in place, parallel work begins.

Estimated duration: 75 minutes (45 to discuss, 30 to type up).

7. Per-person briefings

Six briefings follow. Each is laid out the same way:

- **Mission** — one paragraph: what you are responsible for and why this slice fits your background.
- **Files you own** — the exact paths.
- **Functionalities to implement** — the public interfaces and behaviours.
- **Pydantic models you touch** — the contracts you produce or consume.
- **Dependencies** — who blocks you, who you block.
- **Day 1 / Day 2 / Day 3 plan** — concrete deliverables per day.
- **Definition of done** — how you know your slice is complete.

7.1 Omar — Orchestration & Critic

AI background · conducts the pipeline · owns the agentic loop

Mission

You are the conductor of the agentic loop. Your code calls every other person's code in the right order, threads the trace ID through, manages retries when the critic asks for them, and decides which model tier each request uses. You also write the critic itself — the stage that judges whether the result actually answers the question and emits the confidence score the UI surfaces to the user.

This slice plays to AI expertise because both the orchestrator and the critic are about reasoning over LLM outputs: when to retry, when to give up, how to score confidence, what feedback to send back to the generator. The code is mostly control flow, but the design decisions are deeply about how agentic systems behave.

Files you own

Path	Purpose
src/sql_agent/agents/orchestrator.py	Wires the pipeline; runs the retry loop
src/sql_agent/agents/critic.py	Reviews result; emits confidence score
src/sql_agent/agents/sql_validator.py	sqlglot static checks before execution
src/sql_agent/llm/model_router.py	Routes simple queries to a cheaper model
prompts/critic.md	Critic LLM prompt
docs/decisions/001-multi-stage-agent.md	ADR — you keep updates
tests/unit/test_orchestrator.py	Tests
tests/unit/test_critic.py	Tests
tests/unit/test_model_router.py	Tests
tests/unit/test_sql_validator.py	Tests

Functionalities to implement

- **Orchestrator.run(question) -> QueryResponse** — the top-level entry point called by the FastAPI route. Threads a trace ID through every stage. Catches stage-level exceptions and converts them into typed error responses the UI can render.
- **Retry loop** — max AGENT_MAX_RETRIES (default 3). On each retry, the critic's feedback is appended to the SQL generator's prompt context. The trace ID is preserved so the request is debuggable end-to-end.

- **Model router** — rule-based classifier: questions with no joins, no aggregations beyond COUNT/SUM, and no time-bucketing go to the small model; everything else goes to the large model. The chosen model id is included in CandidateSQL so we can report token cost per tier in the slides.
- **Critic** — an LLM call (using prompts/critic.md) that returns: decision (accept / retry), free-text feedback for the generator, and a confidence score in [0, 1]. Confidence inputs to weight: did all WHERE filter values exist in the data, did the SQL touch the planner's expected tables, does row count look plausible.
- **SQL Validator** — sqlglot static checks: syntactically valid, referenced tables and columns exist, no DDL/DML keywords, complexity caps (max joins, no Cartesian products). The validator runs before the executor so we never waste a DB round-trip on broken SQL.
- **Confidence threshold** — if the final critic confidence is below CONFIDENCE_WARNING_THRESHOLD (default 0.7), the QueryResponse has a warning field set. Mehdi's frontend renders this as a yellow banner.
- **Tracing** — every stage call is wrapped to emit a structured log line with the trace ID, stage name, latency, and outcome. This is the foundation for the observability story.

Pydantic models you produce or consume

Produces: QueryResponse (top-level), CritiqueResult, ValidationResult, ModelTier. **Consumes:** every other model in the pipeline.

Dependencies

You depend on: Hasan (planner, generator, LLM client) for the stages you call; Zayad (schema retriever, few-shot bank, row fallback) for the retrieval calls; Abdul Qayyum (safe executor) to actually run SQL.

You unblock: Mehdi — until orchestrator.run() returns a usable QueryResponse, his API has nothing to call.

Day-by-day plan

Day 1 (2 June, afternoon & evening)

- Lead the morning alignment session on core/models.py.
- Write orchestrator function signatures and stubbed stage calls. Each stage is called with a hardcoded fake response just so the wiring runs.
- Get the trace-ID propagation working — one log line per stage.
- Push core/models.py and an end-to-end-walking orchestrator by end of Day 1.

Day 2 (3 June)

- Replace fake stage calls with real ones as each owner pushes their implementation.
- Implement the critic LLM call with prompts/critic.md. Iterate on the prompt against Asad's golden set.
- Implement the retry loop with critic feedback.

- Implement model routing — rule-based classifier first; if time, add a small-model classification fallback.
- Aim: end-to-end working pipeline by the end of Day 2.

Day 3 (4 June, build to demo)

- Tune retry caps and confidence thresholds based on the benchmark numbers Asad produces.
- Write the orchestrator section of docs/ARCHITECTURE.md (yours is the most visible piece in the demo).
- Lead a 30-minute walkthrough of the pipeline for the team before the demo rehearsal.
- Be the on-stage technical voice in the demo — you understand the loop best.

Definition of done

- Orchestrator runs all seven stages with real implementations.
- Critic emits a confidence score and the retry loop fires when confidence is low.
- Model router demonstrably picks the small model for at least 30% of the golden set.
- All tests in your tests/unit/ files pass.
- Your section of docs/ARCHITECTURE.md is filled in.

7.2 Hasan — Planner & SQL Generator

AI background · the prompt-heavy slice · turns natural language into SQL

Mission

You own the two stages where prompt quality dominates output quality: the planner (deciding whether a question is in scope) and the SQL generator (actually producing the SQL). You also own the LLM client wrapper — the layer between our code and OCI Generative AI — which means everyone else's LLM call goes through your code.

This is the most LLM-iteration-heavy slice in the project. Most of your time will be spent in notebooks/03_prompt_iteration.ipynb tuning prompts against Asad's golden query set, watching the accuracy number climb. Of all the slices, yours has the most direct effect on the headline number we report to the judges.

Files you own

Path	Purpose
src/sql_agent/agents/planner.py	Decides if a question is a data question
src/sql_agent/agents/sql_generator.py	LLM call that produces candidate SQL
src/sql_agent/llm/client.py	OCI Generative AI client wrapper
prompts/planner.md	Planner prompt
prompts/sql_generation.md	SQL generation prompt
prompts/sql_correction.md	Retry prompt with critic feedback
tests/unit/test_planner.py	Tests
tests/unit/test_sql_generator.py	Tests
tests/unit/test_llm_client.py	Tests (mock the OCI call)

Functionalities to implement

- **LLM client** — thin wrapper around OCI Generative AI. `generate(prompt, model_id, max_tokens)` -> `CompletionResult`. Handles auth, retries on transient errors, returns token counts so cost can be reported.
- **Planner** — LLM call (with `prompts/planner.md`) that classifies the question as *data question* / *out-of-scope* / *needs clarification*. Returns a `Plan` with the decision, the scope tags, and (if refused) a refusal reason for the UI to render.
- **SQL generator** — LLM call with `prompts/sql_generation.md`. Assembles the prompt from: the question, the retrieved schema (from Zayad), the few-shot examples (from Zayad), and an optional previous error (from the critic). Returns `CandidateSQL` with the SQL string, the model id used, and the prompt token count.

- **SQL correction path** — when the critic asks for a retry, the generator is called again with prompts/sql_correction.md which includes the previous SQL, the error or critique feedback, and instructions to fix the issue.
- **Prompt iteration** — you'll spend Day 2 iterating prompts against Asad's golden query set. Track each version's accuracy in notebooks/03_prompt_iteration.ipynb — this becomes a methodology slide.
- **Robustness to schema obfuscation** — when SCHEMA_OBFUSCATION_ENABLED is on, the schema you receive has aliased names. Your prompt must not reveal that fact to the LLM — the LLM should just see a normal-looking schema with Table_A, Column_B, etc. The obfuscator (Abdul Qayyum's code) handles the round-trip.

Pydantic models you produce or consume

Produces: Plan, CandidateSQL, CompletionResult. **Consumes:** Question, RetrievedSchema, FewShotExample, CritiqueResult (on retry).

Dependencies

You depend on: Abdul Qayyum (OCI access verified on Day 1 morning), Zayad (schema + few-shot retrieval).

You unblock: Omar (his orchestrator calls your stages); Asad (her benchmark runs your generator).

Day-by-day plan

Day 1 (2 June, afternoon & evening)

- After OCI is verified, smoke-test the LLM client in notebooks/02_test_oci_genai.ipynb. Make one round-trip work end-to-end.
- Implement llm/client.py — the thin wrapper. Push it. Now everyone else's LLM call has somewhere to go.
- Draft prompts/planner.md and prompts/sql_generation.md from scratch. Hardcode a tiny schema to test against.
- Stub planner.py and sql_generator.py with the function signatures.

Day 2 (3 June)

- Implement the planner and generator end-to-end against Zayad's real schema retriever.
- Iterate prompts against Asad's golden query set. Aim for $\geq 70\%$ on easy tier by end of day.
- Implement the SQL correction prompt path. Coordinate with Omar on the feedback format.
- Push your prompt-iteration notebook with measured accuracy per prompt version.

Day 3 (4 June, build to demo)

- Push prompts as high as time allows. Diminishing returns past Day 2 evening — don't over-tune.
- Coordinate with Zayad to test whether dynamic few-shot injection moves the number; A/B if time permits.
- Help Asad write the 'why our pipeline is more accurate' slide. You have the numbers.

Definition of done

- LLM client works against OCI GenAI in production conditions.
- Planner returns sensible Plan objects for at least 5 distinct question types.
- Generator produces valid SQL on $\geq 70\%$ of easy-tier golden queries.
- All prompts are checked in to prompts/; the iteration history is in notebooks/03_prompt_iteration.ipynb.

7.3 Zayad — Retrieval / RAG layer

AI background · the brief's headline requirement · three retrieval surfaces

Mission

RAG is the brief's most explicit requirement. You own all three retrieval surfaces: schema retrieval (finds the right tables for a question), few-shot retrieval (finds similar past queries to inject as examples), and row fallback (finds semantically similar rows when SQL returns zero results). All three share the same vector store infrastructure that Abdul Qayyum builds.

Your slice is the difference between a generic prompt-to-SQL toy and an enterprise-grade text-to-SQL system. Done well, it lets the agent answer questions about a 200-table schema by feeding the LLM only the 5 relevant tables — faster, cheaper, more accurate.

Files you own

Path	Purpose
src/sql_agent/retrieval/schema_retriever.py	Top-k tables for a question
src/sql_agent/retrieval/few_shot_bank.py	Similar past queries from vector store
src/sql_agent/retrieval/row_fallback.py	Similar rows when SQL returns zero
src/sql_agent/retrieval/embeddings.py	OCI Embeddings wrapper
src/sql_agent/llm/prompts.py	Prompt loader/registry
scripts/embed_schema.py	Embed schema descriptions into vector store
docs/decisions/002-schema-retrieval.md	ADR — you keep updates
tests/unit/test_schema_retriever.py	Tests
tests/unit/test_few_shot_bank.py	Tests
tests/unit/test_row_fallback.py	Tests

Functionalities to implement

- **Embeddings wrapper** — calls OCI Generative AI Embeddings (cohere.embed-english-v3.0). `embed(text: str) -> list[float]` and a batch version. Caches embeddings keyed by content hash.
- **Schema retriever** — embeds the user question, queries the vector store for nearest schema descriptions, returns top-k tables with their column descriptions as a `RetrievedSchema`. Default `k = 5`.
- **Few-shot bank** — same flow but against a separate namespace in the vector store, populated from `evaluation/datasets/example_queries.jsonl` (Asad curates). Returns 2-3 examples per call.

- **Row fallback** — called when SQL returns zero rows. Identifies the WHERE filter values from the SQL via sqlglot, embeds them, queries the vector store for nearest matches in the relevant column, returns rows + similarity scores + a flag that the UI uses to show 'approximate match'.
- **Embed-schema script** — reads db/schema_descriptions.yaml (Asad maintains) and pushes embeddings into the vector store. Idempotent — only re-embeds when content hash changes.
- **Prompt registry** — the llm/prompts.py module loads Markdown prompts from prompts/ at startup, validates that referenced template variables exist, returns prompt strings to the agent stages.

Pydantic models you produce or consume

Produces: RetrievedSchema, FewShotExample, RowFallbackResult. **Consumes:** Question, ExecutionResult (for the empty-result trigger).

Dependencies

You depend on: Abdul Qayyum (vector store + Oracle 23ai integration), Asad (schema descriptions YAML + example queries dataset).

You unblock: Hasan (his generator needs the retrieved schema and few-shot examples to write good SQL); Omar (his row fallback hook in the orchestrator).

Day-by-day plan

Day 1 (2 June, afternoon & evening)

- Agree the VectorItem and Hit shapes with Abdul Qayyum during the alignment session.
- Implement embeddings.py using Hasan's LLM client foundation.
- Implement the prompt registry (llm/prompts.py). Hasan's prompts depend on this.
- Stub schema retriever and few-shot bank with the function signatures.

Day 2 (3 June)

- Run embed_schema.py against Asad's schema descriptions. Verify retrieval quality on 5-10 sample questions.
- Implement few-shot bank against Asad's example queries dataset.
- Implement row fallback. Coordinate with Hasan on how the SQL is parsed to extract filter values.
- Tune top-k values per surface. Document recall numbers (e.g. 'top-3 hits the correct table on 90% of golden queries').

Day 3 (4 June, build to demo)

- Run A/B with Hasan: generator accuracy with vs without few-shot injection. The lift is a slide.
- Help Mehdi style the 'approximate match' notice the UI shows when row fallback fires.
- Write the retrieval section of docs/ARCHITECTURE.md. This is where the RAG narrative lives.

Definition of done

- Schema retriever returns the correct table in top-3 for at least 80% of golden queries.

- Few-shot bank returns 2 sensible examples for at least 5 distinct question shapes.
- Row fallback triggers correctly on the zero-row test case and shows approximate matches.
- ADR-002 is updated with the final retrieval design.

7.4 Asad — Summariser, Evaluation & Data Modelling

Data Science (MSc) · the rigour slice · benchmark + summariser + the data layer's semantic features

Mission

You sit at the boundary between data and language. On one side, you curate the golden query set and the example query bank, write the benchmark harness, and produce the headline accuracy number that the judges remember. On the other, you own the summariser — the stage that turns rows of data back into natural language with a plain-English explanation of how the SQL works.

You also own the schema descriptions and the business glossary in db/. This is the ‘feature engineering’ of text-to-SQL: the better the column descriptions, the better the retrieval, the better the SQL. Of all six slices, yours has the highest ceiling on accuracy gains from a few hours of careful data-modelling work.

Files you own

Path	Purpose
src/sql_agent/agents/summariser.py	Rows -> natural-language answer + explanation
src/sql_agent/llm/token_counter.py	Counts and prices tokens per request
src/sql_agent/safety/pii_filter.py	Strips PII from logs and summaries
prompts/summariser.md	Summariser prompt
prompts/sql_explanation.md	Plain-English SQL explanation prompt
evaluation/benchmark.py	Runs agent against golden set, computes metrics
evaluation/metrics.py	Execution accuracy, exact-set, partial match
evaluation/datasets/golden_queries.jsonl	20+ benchmark questions you curate
evaluation/datasets/example_queries.jsonl	10 examples for the few-shot bank
db/schema_descriptions.yaml	Business descriptions per table/column
db/glossary.yaml	Business term synonyms
scripts/preprocess_raw_data.py	Raw -> cleaned seed CSVs
scripts/build_schema_descriptions.py	Syncs YAML with live DDL
scripts/run_benchmark.py	Make-targeted runner

Path	Purpose
notebooks/01_explore_dataset.ipynb	Profile the seed data
notebooks/03_prompt_iteration.ipynb	Track prompt-vs-score curve
docs/EVALUATION.md	Methodology doc — fill in
docs/RELATED_WORK.md	Spider, BIRD, DAIL-SQL citations
tests/unit/test_summariser.py	Tests
tests/unit/test_metrics.py	Tests
tests/unit/test_benchmark.py	Tests

Functionalities to implement

- **Golden query set** — at least 20 questions across easy/medium/hard tiers, with expected SQL and expected rows. This is the single most important dataset in the project — without it, nobody can measure anything.
- **Example query bank** — 8-12 carefully picked examples spanning the SQL patterns the generator needs to learn (joins, aggregates, window functions, date filtering). Disjoint from the golden set.
- **Schema descriptions YAML** — for every table and column in the DDL, write a 1-2 sentence business-language description, plus units, timezones, currencies, and sample values for low-cardinality columns. This is the data the RAG layer embeds.
- **Glossary** — synonyms map ('revenue' = 'sales' = 'turnover' -> orders.total_gbp). Used as additional retrieval signal.
- **Metrics** — execution accuracy (do the generated and expected rows match?), exact-set match, partial match, retry rate, latency p50/p95, token cost per request.
- **Benchmark harness** — runs the agent end-to-end against the golden set, writes a timestamped JSON to evaluation/results/runs/, prints a pass/fail report. Used hourly during Day 2.
- **Summariser** — LLM call that turns rows + SQL + question into: a one-sentence direct answer, a 2-4 bullet plain-English explanation, and a list of tables used. Two prompts: summariser.md for the answer, sql_explanation.md for the explanation panel.
- **PII filter** — strips email addresses, phone numbers, and similar from log lines and the summariser output. Small but important — required by the safety story.
- **Preprocessing** — if the demo dataset comes as raw CSVs that need cleaning, this is your script. data/raw/ -> db/seed/.

Pydantic models you produce or consume

Produces: AnswerSummary, BenchmarkResult, Metric. **Consumes:** Question, CandidateSQL, ExecutionResult, RetrievedSchema (for tables_used).

Dependencies

You depend on: Abdul Qayyum (DDL must exist before you can describe the schema; DB must be seeded before you can benchmark); Hasan (your benchmark runs his generator).

You unblock: Zayad (your schema descriptions are what he embeds; your example bank is what his few-shot retriever returns); Hasan (your golden set is what he optimises his prompts against); the slide deck (your numbers are the demo).

Day-by-day plan

Day 1 (2 June, afternoon & evening)

- After Abdul Qayyum pushes the DDL, draft `db/schema_descriptions.yaml` and `db/glossary.yaml`.
- Curate at least 10 golden queries against the seed schema. Write the expected SQL for each.
- Stub the metrics functions and benchmark harness with the right signatures.
- Profile the seed data in `notebooks/01_explore_dataset.ipynb` — column cardinalities, null rates, ranges. This informs what sample values to include in the descriptions.

Day 2 (3 June)

- Implement the metrics functions and the full benchmark harness. Run it hourly — the numbers drive Hasan's prompt iteration.
- Expand the golden set to 20+ queries.
- Curate the example query bank for the few-shot retriever.
- Implement the summariser end-to-end. Iterate the `summariser.md` and `sql_explanation.md` prompts on a small sample.
- Fill in `docs/EVALUATION.md` with the metrics methodology.

Day 3 (4 June, build to demo)

- Run the final benchmark. Produce the headline accuracy number, the per-tier breakdown, the retry rate, the cost-per-query.
- Make the 'numbers' slide for the deck — this is what the judges remember.
- Fill in `docs/RELATED_WORK.md` with Spider / BIRD / DAIL-SQL citations and how our pipeline compares.
- Update `CHANGELOG.md` with the final result.

Definition of done

- 20+ golden queries with expected SQL.
- Schema descriptions cover every table and column in the DDL.
- Benchmark runs end-to-end and writes a results JSON.
- Summariser produces sensible answers and explanations on the golden set.
- `docs/EVALUATION.md` and `docs/RELATED_WORK.md` filled in.
- Final benchmark number ready to slide.

7.5 Mehdi — FastAPI & Streamlit

CS (undergrad) · the product-facing slice · what the judges actually see

Mission

You own the two surfaces that turn the backend pipeline into a demo-able product: the FastAPI service that exposes the agent over HTTP, and the Streamlit frontend the user (and the judges) actually interact with. Everyone else's work is invisible without yours.

Your slice is bounded and well-understood engineering work that maps cleanly onto a CS portfolio story. The interesting work is in the four UI surfaces: the confidence-warning banner that surfaces Omar's score; the 'How did the AI calculate this?' expander that shows Asad's explanation; the friendly error states that catch every typed exception from `core/exceptions.py`; and the approximate-match notice when Zayad's row fallback fires.

Files you own

Path	Purpose
<code>src/sql_agent/api/main.py</code>	FastAPI application factory
<code>src/sql_agent/api/routes.py</code>	The <code>/query</code> endpoint and error handlers
<code>src/sql_agent/api/schemas.py</code>	HTTP request/response Pydantic models
<code>frontend/streamlit_app.py</code>	The four UI surfaces
<code>Dockerfile</code>	Verify image builds and runs
<code>docker-compose.yml</code>	Local stack — verify it boots
<code>.github/workflows/ci.yml</code>	Keep it green
<code>tests/unit/test_api_routes.py</code>	Route tests
<code>tests/integration/test_end_to_end.py</code>	End-to-end test
<code>deploy/README.md</code>	Fill in the deployment runbook

Functionalities to implement

- **POST `/query`** — accepts `QueryRequest` (just `{question: str}`), calls `orchestrator.run()`, returns `QueryResponse`. Plus health-check, version, and a debug endpoint that exposes the last trace.
- **Friendly error mapping** — every typed exception from `core/exceptions.py` is caught and mapped to a friendly user message. Never a raw 500 with a traceback. See the error-paths table in `docs/REQUEST_LIFECYCLE.md`.
- **Streamlit UI — question/answer panel** — text input, submit button, direct answer at the top, result table below.

- **Streamlit UI — confidence indicator** — green badge if confidence $\geq$ threshold; yellow banner with the message 'I generated this answer, but the schema mapping was ambiguous. Please double-check before acting on it' otherwise.
- **Streamlit UI — SQL Explainer expander** — collapsible panel titled 'How did the AI calculate this?'. Shows: the SQL (syntax-highlighted), the plain-English explanation from the summariser, the list of tables used.
- **Streamlit UI — friendly error states** — out-of-scope, refused, retry-exhausted, timeout. Each has its own copy block; all copy lives at the top of `streamlit_app.py` for easy edits.
- **Streamlit UI — approximate-match notice** — when `RowFallbackResult` is present in the response, render a notice above the table: 'No exact match for X. Showing closest results instead'.
- **Demo cache toggle** — a hidden UI control (or just an env-var flag) flips `DEMO_FALLBACK_ENABLED=true`. Use only for the demo. Tested in rehearsal.
- **Docker stack** — verify docker compose up brings up API + UI together. The Streamlit container talks to the API container by service name.

Pydantic models you produce or consume

Produces: `QueryRequest`, `QueryResponse`, plus HTTP error types. **Consumes:** `AnswerSummary` (the payload of `QueryResponse`).

Dependencies

You depend on: Omar — until `orchestrator.run()` returns something, your API has nothing to wrap; Asad — for the `AnswerSummary` shape; Abdul Qayyum — for the demo cache wiring.

You unblock: the demo. Everyone else's work is invisible without your frontend.

Day-by-day plan

Day 1 (2 June, afternoon & evening)

- Stand up a `/query` endpoint that returns a hardcoded fake `QueryResponse`. This unblocks Omar's wiring AND gives you a target to point Streamlit at.
- Stand up the Streamlit app showing the four UI surfaces against the fake response. Polish the copy.
- Verify docker compose up brings both up together.
- Push everything — the team can now see what they're building.

Day 2 (3 June)

- Replace the fake response with the real orchestrator call as soon as Omar pushes a working pipeline.
- Implement the error-state mapping for every exception in `core/exceptions.py`.
- Wire up the confidence indicator and the approximate-match notice.
- Polish the SQL Explainer panel — syntax highlighting, table list, explanation rendering.

Day 3 (4 June, build to demo)

- Demo-rehearse with Asad. Walk through the three demo questions end-to-end. Time it.
- Wire up the demo cache toggle. Test the failover by manually killing the DB container.
- Take final screenshots for the slide deck.
- On demo day: run the demo. Be ready to flip the demo cache flag if anything wobbles.

Definition of done

- Streamlit + FastAPI run together via docker compose up.
- All four UI surfaces work against the real backend.
- Every typed exception maps to a friendly error message.
- Demo cache failover tested in rehearsal.
- deploy/README.md filled in with the deployment runbook.

7.6 Abdul Qayyum — Database, Vector Store & Safety

CS (undergrad) · the foundation slice · the engineering everyone depends on

Mission

You own the foundation: the Oracle Autonomous Database integration, the Oracle 23ai vector store, and the safety layer that protects the database from anything the LLM produces. If your work isn't ready by Day 1 afternoon, nobody else can integrate end-to-end — which is why your morning task is the critical path of the whole project.

Your slice is concrete Oracle engineering with three sharp deliverables and clear acceptance criteria. The schema obfuscator is a fun stretch: it's pure string manipulation that lets the system claim enterprise-grade privacy, which judges love.

Files you own

Path	Purpose
src/sql_agent/database/connection.py	Connection pool to Autonomous DB
src/sql_agent/database/safe_executor.py	Read-only, timeout, row cap
src/sql_agent/database/demo_cache.py	Fail-safe fallback cache
src/sql_agent/database/schema_introspector.py	Read DB schema at runtime
src/sql_agent/retrieval/vector_store.py	Oracle 23ai VECTOR integration
src/sql_agent/safety/sql_guard.py	sqlglot guard: no DDL/DML, joins capped
src/sql_agent/safety/schema_obfuscator.py	Real names <-> aliases round-trip
db/ddl/01_create_tables.sql	Initial schema DDL
db/migrations/	Versioned schema changes
scripts/seed_database.py	Apply DDL + load seed CSVs
scripts/embed_schema.py	Push descriptions into vector store
docs/decisions/003-safety-guardrails.md	ADR — you keep updates
docs/diagrams/er_diagram.md	Keep in sync with DDL
notebooks/02_test_oci_genai.ipynb	OCI access verification — Day 1 critical
tests/unit/test_safe_executor.py	Tests
tests/unit/test_sql_validator.py	Tests

Path	Purpose
tests/unit/test_vector_store.py	Tests
tests/unit/test_schema_obfuscator.py	Tests

Functionalities to implement

- **OCI access verification** — the first 30 minutes of Day 1. In notebooks/02_test_oci_genai.ipynb, prove that we can call OCI GenAI and connect to Autonomous DB. If this fails, the whole project stalls — surface it loudly.
- **DDL** — write the demo schema in db/ddl/01_create_tables.sql with PKs, FKs, indexes, and COMMENT clauses on every column.
- **seed_database.py** — applies the DDL, loads db/seed/*.csv into the right tables, creates the read-only DB user the agent uses.
- **Connection pool** — managed pool over oracledb. Configurable size from settings. Reused across requests.
- **Safe executor** — runs SQL against the read-only user, enforces 15-second timeout, 500-row cap, returns ExecutionResult. Checks demo_cache first if the flag is on; falls back to the cache on DB error.
- **Schema introspector** — reads the live DB schema (tables, columns, types, FKs) at startup so the validator can check that LLM-generated SQL references real columns.
- **Vector store** — thin abstraction over Oracle 23ai's VECTOR datatype. Supports two namespaces (schema descriptions, example queries) with cosine similarity search.
- **SQL guard** — sqlglot-based: rejects DDL, rejects DML (INSERT/UPDATE/DELETE), caps joins, rejects unbounded Cartesian products.
- **Schema obfuscator** — replaces real table and column names with Table_A, Column_1 aliases before the schema is sent to the LLM, returns an AliasMap that's used to translate the generated SQL back. Round-trip integrity is the test.
- **Demo cache** — populated from a rehearsal run on Day 3 morning. Stored in evaluation/datasets/demo_cache.json.

Pydantic models you produce or consume

Produces: ExecutionResult, VectorItem, Hit, AliasMap. **Consumes:** ValidatedSQL, RetrievedSchema (for obfuscation).

Dependencies

You depend on: nobody — you are upstream of everyone.

You unblock: Hasan (no LLM without OCI access); Zayad (no vector store, no retrieval); Asad (no DB, no benchmark); Omar (no executor, no pipeline); Mehdi (no backend, no API).

Day-by-day plan

Day 1 (2 June, afternoon & evening)

- **Critical path:** 9-11 AM: verify OCI access works in notebooks/02_test_oci_genai.ipynb. If it doesn't, raise the alarm immediately.
- 11 AM-1 PM: write db/ddl/01_create_tables.sql and run seed_database.py against the Autonomous DB.
- Afternoon: implement connection.py and a minimal safe_executor.py. Push by end of Day 1.
- Evening: stub the vector store and schema obfuscator.

Day 2 (3 June)

- Implement the full vector store. Coordinate with Zayad on the VectorItem / Hit shapes.
- Run embed_schema.py against Asad's schema descriptions YAML.
- Implement the schema obfuscator. Round-trip test against the validator to make sure aliases never leak into the executed SQL.
- Implement the SQL guard. Coordinate with Omar on which patterns to reject.

Day 3 (4 June, build to demo)

- Populate demo_cache.json from a rehearsal run with the three demo questions.
- Test the failover: kill the DB connection, verify the cache serves the result.
- Write the database / safety section of docs/ARCHITECTURE.md.
- Update ADR-003 with the final safety design.

Definition of done

- OCI access verified, DB seeded, descriptions embedded.
- Safe executor enforces all three limits (read-only, timeout, row cap) under test.
- Schema obfuscator round-trips cleanly for every test schema.
- SQL guard rejects every test case in tests/unit/test_sql_validator.py.
- Demo cache failover tested in rehearsal.

8. Integration points

These are the seams where two people's code meets. Each one is a potential 3 AM crisis if the contract isn't clear. The contracts are in `core/models.py` — this section describes the spots where two people need to talk before pushing.

Seam	Owners	What to agree on
VectorItem / Hit shapes	Abdul Qayyum ↔ Zayad	Embedding dimension, namespace string, metadata keys
AliasMap round-trip	Abdul Qayyum ↔ Hasan	How the LLM sees the schema; how SQL is de-obfuscated
RetrievedSchema shape	Zayad ↔ Hasan	Column descriptions, sample values, FK hints
FewShotExample shape	Zayad ↔ Hasan	What rationale field looks like
Critic feedback format	Omar ↔ Hasan	How feedback strings are injected into the correction prompt
Confidence threshold	Omar ↔ Mehdi	When the UI banner triggers
ExecutionResult on zero rows	Abdul Qayyum ↔ Zayad	How the row fallback hook is triggered
AnswerSummary fields	Asad ↔ Mehdi	What the explainer panel renders
QueryResponse error fields	Omar ↔ Mehdi	Mapping <code>core/exceptions.py</code> to UI copy fields
Golden query expected_sql	Asad ↔ Abdul Qayyum	DDL must match what the expected SQL queries

If you find yourself blocked at one of these seams, the protocol is: post in the group chat, tag the other owner, and propose the contract you want. If you don't get a reply in 15 minutes, ping by voice. Don't wait silently — the project moves faster than that.

9. Request lifecycle

End-to-end trace of one user question, with owners at every step. The full text version with error paths is in docs/REQUEST_LIFECYCLE.md.

FRONTEND (Mehdi)

1. User types question in Streamlit
2. POST /query -> FastAPI

BACKEND API (Mehdi)

3. routes.py validates payload with QueryRequest
4. Calls orchestrator.run(question)

ORCHESTRATOR (Omar)

- | | | |
|---|-----------|---------------------|
| 5. Planner.plan(question) | [Hasan] | -> Plan |
| 6. SchemaRetriever.retrieve(question) | [Zayad] | -> RetrievedSchema |
| 7. FewShotBank.get_examples(question) | [Zayad] | -> [FewShotExample] |
| 8. SchemaObfuscator.obfuscate(schema) | [Abdul Q] | -> AliasMap |
| 9. ModelRouter.pick_model(question, plan) | [Omar] | -> model_id |
| 10. SQLGenerator.generate(...) | [Hasan] | -> CandidateSQL |
| 11. SchemaObfuscator.deobfuscate(sql) | [Abdul Q] | -> real SQL |
| 12. SQLValidator.validate(sql, schema) | [Omar] | -> ValidationResult |
| 13. SafeExecutor.execute(sql) | [Abdul Q] | -> ExecutionResult |
| 14. (if 0 rows) RowFallback.find_similar(...) | [Zayad] | -> approximate rows |
| 15. Critic.review(...) | [Omar] | -> CritiqueResult |
| | | |
| +--- retry up to N times -> back to step 10 | | |
| +--- accept -> step 16 | | |
| 16. Summariser.summarise(...) | [Asad] | -> AnswerSummary |

BACKEND API (Mehdi)

17. Wrap AnswerSummary in QueryResponse, return JSON

FRONTEND (Mehdi)

18. Render answer, confidence badge, table, SQL Explainer, approximate-match notice

10. Day-by-day timeline

Day 1 — 2 June (Tuesday)

Time	What happens	Lead
09:00	Kickoff briefing (hackathon organisers)	All
10:00	Team meets — 75-minute alignment session on core/models.py	Omar
11:15	Critical path: OCI access verification + DB seeding	Abdul Qayyum
11:15	Parallel: prompt drafting + LLM client skeleton	Hasan
11:15	Parallel: API + Streamlit stub against fake response	Mehdi
11:15	Parallel: schema descriptions YAML + golden set draft	Asad
11:15	Parallel: embeddings + prompt registry stubs	Zayad
13:00	Standup #1 + lunch	All
14:00	Continue afternoon work — all slices in progress	All
17:00	Standup #2	All
20:00	End-of-Day-1 target: minimal end-to-end (with fakes) running	All
22:00	Sleep. Seriously.	All

Day 2 — 3 June (Wednesday)

Time	What happens	Lead
09:00	Standup #3 — integration day plan	Omar
09:30	Real implementations replace stubs across the board	All
13:00	Standup #4 — first benchmark run	Asad
14:00	Prompt iteration begins, drives accuracy curve	Hasan + Asad
17:00	Standup #5	All
20:00	End-of-Day-2 target: full pipeline working on golden set	All
20:00	Benchmark numbers locked in for the slide	Asad
22:00	Sleep. Seriously.	All

Day 3 — 4 June (Thursday, demo day)

Time	What happens	Lead
09:00	Standup #6 — demo prep plan	Omar
09:30	Demo cache populated from rehearsal	Abdul Qayyum + Mehdi
10:00	Slide deck assembly	Asad + Omar
11:00	Demo rehearsal #1 — end to end	All
12:00	Standup #7 — final fixes	All
13:00	Demo rehearsal #2 — timed	All
14:30	Slides locked, demo locked. Pack up.	All
15:00	Demo to judges (timing TBC)	Omar + Asad + Mehdi on stage
evening	Closing ceremony, networking with Enigen + Oracle	All

11. Demo plan

Five minutes on stage. Three demo questions, one safety net, one strong narrative. The structure below is a starting point — the team rehearses it twice on Day 3.

Structure (5 minutes)

- 1 **Open with the problem** (30s) — one slide. Every company has analysts manually translating questions into SQL. Show the cost.
- 2 **The architecture diagram** (45s) — one slide showing the 7-stage pipeline. Highlight: critic-driven retry, RAG over schema, multi-model routing.
- 3 **Live demo, question 1** (60s) — an easy query. Show the answer, the result table, and crucially the 'How did the AI calculate this?' expander.
- 4 **Live demo, question 2** (60s) — a medium-complexity query (multi-table join). Point out the schema retrieval — only 3 tables sent to the LLM out of N.
- 5 **Live demo, question 3** (60s) — the hard one, where the row fallback fires (deliberately ask about a value that's misspelled). Show the approximate-match notice.
- 6 **The numbers slide** (45s) — benchmark accuracy, retry rate, cost-per-query, latency.
- 7 **One closing slide** (30s) — the brief's three requirements ticked, plus our six innovations.

Safety net

Five minutes before going on stage, DEMO_FALLBACK_ENABLED is flipped to true. If anything wobbles — the network, OCI rate-limiting, the DB — the cache serves the answer instantly. The judges see a clean demo. Nobody explains the cache.

Roles on stage

- **Omar** — the technical voice. Explains the architecture, talks through the demo.
- **Asad** — the numbers voice. Walks through the benchmark slide and the methodology.
- **Mehdi** — the operator. Types the questions, controls the UI.
- **Everyone else** — in the audience, ready for Q&A.

12. Appendices

A. File ownership matrix

Path	Owner
src/sql_agent/core/models.py	All (Day 1 alignment) / Omar (custodian)
src/sql_agent/core/exceptions.py	Omar
src/sql_agent/core/logging.py	Omar
src/sql_agent/config/settings.py	Whoever adds the env var; Omar reviews
src/sql_agent/llm/client.py	Hasan
src/sql_agent/llm/model_router.py	Omar
src/sql_agent/llm/prompts.py	Zayad
src/sql_agent/llm/token_counter.py	Asad
src/sql_agent/retrieval/*	Zayad
src/sql_agent/retrieval/vector_store.py	Abdul Qayyum
src/sql_agent/database/*	Abdul Qayyum
src/sql_agent/safety/sql_guard.py	Abdul Qayyum
src/sql_agent/safety/pii_filter.py	Asad
src/sql_agent/safety/schema_obfuscator.py	Abdul Qayyum
src/sql_agent/agents/planner.py	Hasan
src/sql_agent/agents/sql_generator.py	Hasan
src/sql_agent/agents/sql_validator.py	Omar
src/sql_agent/agents/critic.py	Omar
src/sql_agent/agents/orchestrator.py	Omar
src/sql_agent/agents/summariser.py	Asad
src/sql_agent/api/*	Mehdi
frontend/streamlit_app.py	Mehdi
prompts/planner.md	Hasan

Path	Owner
prompts/sql_generation.md	Hasan
prompts/sql_correction.md	Hasan
prompts/critic.md	Omar
prompts/summariser.md	Asad
prompts/sql_explanation.md	Asad
db/ddl/*	Abdul Qayyum
db/schema_descriptions.yaml	Asad
db/glossary.yaml	Asad
evaluation/*	Asad
scripts/seed_database.py	Abdul Qayyum
scripts/embed_schema.py	Zayad
scripts/run_benchmark.py	Asad
scripts/preprocess_raw_data.py	Asad
scripts/build_schema_descriptions.py	Asad
notebooks/01_explore_dataset.ipynb	Asad
notebooks/02_test_oci_genai.ipynb	Abdul Qayyum
notebooks/03_prompt_iteration.ipynb	Hasan + Asad
docs/ARCHITECTURE.md	Each owner fills their section
docs/EVALUATION.md	Asad
docs/RELATED_WORK.md	Asad
docs/decisions/001-multi-stage-agent.md	Omar
docs/decisions/002-schema-retrieval.md	Zayad
docs/decisions/003-safety-guardrails.md	Abdul Qayyum
docs/diagrams/er_diagram.md	Abdul Qayyum
CHANGELOG.md	Asad (evening updates)

B. Environment setup checklist

Each person runs through this once after cloning, ideally before Day 1.

```
# Clone
git clone <repo-url>
cd IADS-Agentic-SQL-Agent

# Set up Python 3.11 venv
python3.11 -m venv .venv
source .venv/bin/activate          # macOS / Linux
.venv\Scripts\activate             # Windows

# Install everything + register pre-commit hooks
make dev

# Configure secrets
cp .env.example .env
# Edit .env with your OCI compartment ID, region, model IDs, ADB credentials

# Sanity check
make lint
make test
make run-api      # in one terminal
make run-ui       # in another

# When you finish a chunk
git add -p
git commit -m 'feat(scope): description'
git push origin feat/<name>--<module>
```

C. Cheat sheet

Want to...	Run...
Install everything	make dev
Run linter	make lint
Run tests	make test
Run the API locally	make run-api
Run the Streamlit UI	make run-ui
Apply DDL + load seed	make seed-db
Refresh schema descriptions	make refresh-schema-descriptions
Embed schema into vector store	make embed-schema
Run the benchmark	make benchmark
Bring up the full docker stack	make docker-up
Stop the docker stack	make docker-down
Preprocess raw data	make preprocess

D. Communication

- **Group chat** — everything goes here. Block / unblock signals, decisions, links to PRs.
- **Standups** — every 4 hours during the hackathon. 5 minutes max. Omar runs them.
- **1:1 ping** — for tightly coupled work (Hasan ↔ Zayad on retrieval shape, Abdul Qayyum ↔ Zayad on vector store). Loop in the group chat afterwards.
- **If you get stuck** — 15 minutes is the rule. After 15 minutes of being stuck, ask. The hackathon moves too fast for silent struggle.

This document is a living artefact. If something in here turns out to be wrong, fix it on a branch and open a PR — same conventions as code. The most current version always lives in the repository at docs/OWNERSHIP.md, docs/STRUCTURE.md, and docs/REQUEST_LIFECYCLE.md.